Data Visualization and XLSX to CSV Converter Application

This application serves as an interactive tool for data visualization and file conversion. Built using **Streamlit**, it allows users to upload CSV files for visualization, apply dynamic filters, and customize plots. Additionally, it supports converting XLSX files to CSV format for easy manipulation and download.

Main Features

- Upload and visualize CSV data using interactive plots.
- Filter data dynamically through a sidebar.
- Convert Excel (XLSX) files to CSV format.
- Edit datasets and download the modified version.

Key Libraries Used

- *Streamlit*: Provides a framework for creating an interactive web app.
- *Pandas*: Used for data manipulation and analysis.
- *chardet*: Detects file encoding for proper file reading.
- *Plotly*: Generates interactive visualizations.
- *Xlsx2csv*: Converts Excel files into CSV format.

Supported plot types

Plot Type	Data Type	Description
Histogram	Numeric	Displays the distribution of numeric data by grouping into bins.
Box Plot	Numeric	Shows the distribution of data based on five summary statistics (min, Q1, median, Q3, max).
Boxen Plot	Numeric	A variation of a box plot, showing more details about the tails of the distribution.
Heatmap	Numeric	Displays the data values using color scales in a matrix format.
Bubble Chart	Numeric	A scatter plot with an additional size variable to represent another dimension.
Density Plot	Numeric	Shows the distribution of numeric data using a density contour plot.
Violin Plot	Numeric	Combines a box plot with a kernel density estimate, showing data distribution.
3D Scatter Plot	Numeric	A scatter plot in three dimensions (X, Y, Z axes).
Bar Plot	Numeric	Displays data using bars, where the length of each bar is proportional to the value.

Plot Type	Data Type	Description
Bar Chart	Categorical	Displays data using grouped bars for each category.
Pie Chart	Categorical	Shows parts of a whole as slices of a pie.
Sunburst	Categorical	Represents hierarchical categories in a circular diagram.
Dot Plot	Categorical	Plots dots along an axis for each category to show frequency or value.
Heatmap	Categorical	Displays categorical relationships in a matrix using color.
Treemap	Categorical	Displays hierarchical data as nested rectangles.

Code Breakdown

1) Detect File Encoding

```
def detect_encoding(file):
    raw_data = file.read()
    result = chardet.detect(raw_data)
    return result['encoding']
```

This function determines the encoding of the uploaded file using the chardet library. It reads the raw file content, detects the encoding, and returns it. This ensures the file is read correctly in the next steps.

- **Purpose**: Detects the encoding of uploaded files using chardet.
- **Steps**:
  a) Reads the raw file data.
  b) Uses chardet.detect() to find the file's encoding.
  c) Returns the detected encoding.

2) Session State Initialization

```
if 'plots' not in st.session_state:
    st.session_state.plots = []
```

This initializes a session state variable plots if it doesn't already exist, ensuring the generated plots are retained across user interactions.

- **Purpose**: Ensures that a session state variable (plots) exists to store generated plots across user interactions.

3) Generate Numeric Plots

```
def generate_numeric_plot(df, x_axis, y_axis, plot_type, size_var=None, color_var=None,
z_axis=None):
    if plot_type == "Histogram":
        fig = px.histogram(df, x=x_axis, color=x_axis)
    # More conditions follow for other plot types (Box Plot, Heatmap, etc.)
    return fig
```

This function generates numeric-based plots like histograms or box plots using the Plotly Express (px). It constructs different types of plots based on the user's selected plot_type, x_axis, y_axis, and other optional parameters (e.g., size, color). The resulting Plotly figure is returned for display.

- **Purpose**: Dynamically generates various numeric plots based on user input.

- **Steps**:
    1. Uses Plotly Express (px) to generate a plot.
    2. Supports several plot types (e.g., Histogram, Box Plot, Heatmap).
    3. Returns the generated Plotly figure.

4) Generate Categorical Plots

```
def generate_categorical_plot(df, x_axis, y_axes, plot_type):
    if plot_type == "Bar Chart":
        fig = go.Figure()
        for y_axis in y_axes:
            fig.add_trace(go.Bar(x=df[x_axis], y=df[y_axis], name=y_axis))
    return fig
```

This function handles the generation of categorical plots (e.g., bar charts, pie charts). It uses go.Figure() from Plotly to create a plot, adding multiple series (y_axes) for the selected x_axis. The result is a Plotly figure that is rendered for visualizing categorical data.

- **Purpose**: Generates categorical plots, like Bar Charts or Pie Charts.
- **Steps**:
    1. Supports multiple plot types for categorical data.
    2. Iterates over y_axes to add multiple series to a single plot.
    3. Returns the figure.

5) Delete Plot

```
def delete_plot(index):
    if 0 <= index < len(st.session_state.plots):
        st.session_state.plots.pop(index)
```

This function removes a plot from the session state based on the provided index. It ensures that the selected plot is deleted from the app's state, thus removing it from the display.

- **Purpose**: Deletes a plot from session state by index.
- **Steps**:
    1. Checks if the index is valid.
    2. Removes the plot from st.session_state.plots.

6) **Page Configuration**

```
st.set_page_config(page_title="Data Visualizer & Converter", layout="wide",
page_icon="📊")
```

This configures the Streamlit page layout, setting a title, icon, and a wide layout for better visualization.

- **Purpose**: Configures the Streamlit page with a title, layout, and icon.

7) **Sidebar for Page Selection**

```
page = st.sidebar.selectbox("Select Page", ["Data Visualizer", "XLSX to CSV Converter"])
```

The sidebar provides a dropdown to switch between the "Data Visualizer" and "XLSX to CSV Converter" pages. The selected page is stored in the page variable to control which functionality to display.

- **Purpose**: Allows users to toggle between different pages ("Data Visualizer" and "XLSX to CSV Converter").

8) **Data Visualizer Page**

```
if page == "Data Visualizer":
    st.title("📊 Advanced Data Visualizer")
    uploaded_file = st.sidebar.file_uploader("Choose a CSV file", type=['csv'])

    if uploaded_file is not None:
        file_encoding = detect_encoding(uploaded_file)
        df = pd.read_csv(uploaded_file, encoding=file_encoding)
        # Filtering and plot generation logic continues
```

This section presents the data visualizer interface, allowing users to upload a CSV file, filter data, and generate plots. Once a file is uploaded, the app reads it into a Pandas dataframe and lets users manipulate and visualize the data.

- **Purpose**: Handles the core functionality of the data visualizer.
- **Steps**:
    1. Accepts a CSV file upload.
    2. Detects encoding using detect_encoding().
    3. Reads the file into a Pandas dataframe.
    4. Allows the user to filter the dataset and generate plots.

9) **Dynamic Filtering**

```
if categorical_columns:
    st.sidebar.subheader("Dynamic Filters")
    for col in categorical_columns:
        deselected_values = st.sidebar.multiselect(f"Deselect options from {col}",
options=unique_values)
        filtered_df = filtered_df[filtered_df[col].isin(selected_values)]
```

This dynamically filters categorical data based on the user's selection. The user can deselect values from each categorical column, which filters the dataframe to only include the selected values.

- **Purpose**: Enables users to dynamically filter data by deselecting values from categorical columns.

10) **Plot Selection and Generation**

```
python
Copy code
chart_type = st.selectbox("Choose plot type", chart_options)

if st.button("Generate Plot"):
    if chart_type in ["Histogram", "Box Plot", ...]:
        fig = generate_numeric_plot(filtered_df, x_axis, y_axis, chart_type)
    elif chart_type in ["Bar Chart", "Pie Chart"]:
        fig = generate_categorical_plot(filtered_df, x_axis, y_axes, chart_type)
```

This part of the code lets users choose a plot type and axes for plotting. Depending on the plot type (numeric or categorical), it calls the appropriate function (generate_numeric_plot or generate_categorical_plot) to generate and display the plot.

- **Purpose**: Displays UI for selecting a chart type and axes. On button click, generates and stores the plot in session state.

11) **Plot Display and Delete Button**

```
for i, plot_data in enumerate(st.session_state.plots):
    st.plotly_chart(plot_data["plot"])
    st.button(f"Delete the plot {plot_data['plot_type']}", on_click=delete_plot, args=(i,))
```

The generated plots are displayed in a loop, and each plot has a corresponding delete button. When clicked, the delete function removes the plot from session state.

- **Purpose**: Displays all generated plots and provides a button to delete each.

12) XLSX to CSV Converter Page

```
if page == "XLSX to CSV Converter":
    uploaded_file = st.file_uploader("Choose an XLSX file", type=["xlsx"])

    if uploaded_file is not None:
        xlsx2csv = Xlsx2csv(uploaded_file, delimiter=',', outputencoding='utf-8')
        sheet_name = st.selectbox("Select sheet to convert:", sheet_names)
        xlsx2csv.convert(output_csv, sheetid=sheet_id)
        st.download_button(label="Download Edited CSV", data=edited_csv,
file_name="converted.csv")
```

This section handles XLSX file uploads. After a file is uploaded, the Xlsx2csv library is used to convert a selected sheet into CSV format, which the user can download.

- **Purpose**: Converts an uploaded XLSX file to CSV and allows users to download the converted file.